Arab Academy for Science , Technology & Maritime Transport
College of Computing and Information Technology

Introduction to Problem Solving & Programming (CS143)

Section 13 :
Searching & Sorting

Eng. Mahmoud Osama Radwan
mhmoudko@msn.com

PART1

Searching Algorithms

A typical searching algorithm over an array returns the array index where the target item is found, or –1 if it is not found

```
x[] = {5 , 4 , 3 , 2 , 1}
```

Search for 3 => returns 2

Search for 7 => returns -1

We will examine two specific algorithms: Linear search , Binary search

Linear Search

Algorithm

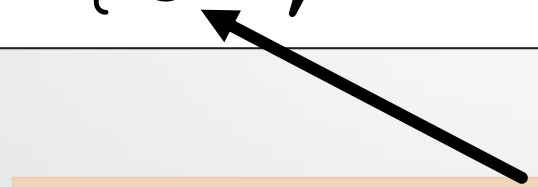
```
found = -1
For i=0 … n
  if(x[i]==target)
    found = i
    break
```

look at each item in turn, beginning with the first one, until either you find the target item or you reach the end of the array

Linear Search : example

Target is 2

```
x[] = {5 , 4 , 3 , 2 , 1}
```



5 == target ?

no

Linear Search : example

Target is 2

```
x[] = {5 , 4 , 3 , 2 , 1}
```

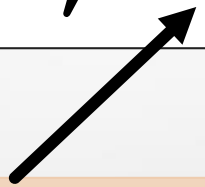
4 == target ?

no

Linear Search : example

Target is 2

```
x[] = {5 , 4 , 3 , 2 , 1}
```



3 == target ?

no

Linear Search : example

Target is 2

```
x[] = {5 , 4 , 3 , 2 , 1}
```

2 == target ?

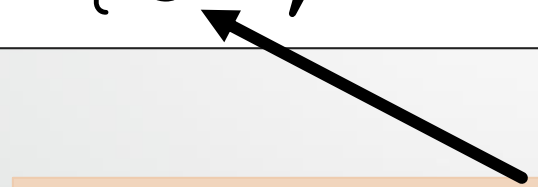
Yes

print 3

Linear Search : example

Target is 7

```
x[] = {5 , 4 , 3 , 2 , 1}
```



5 == target ?

no

Linear Search : example

Target is 7

```
x[] = {5 , 4 , 3 , 2 , 1}
```

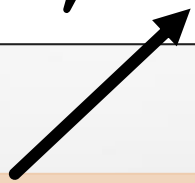
4 == target ?

no

Linear Search : example

Target is 7

```
x[] = {5 , 4 , 3 , 2 , 1}
```



3 == target ?

no

Linear Search : example

Target is 7

```
x[] = {5 , 4 , 3 , 2 , 1}
```

2 == target ?

no

Linear Search : example

Target is 7

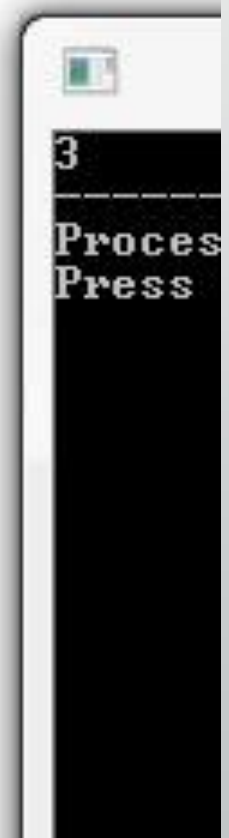
```
x[] = {5 , 4 , 3 , 2 , 1}
```

1 == target ?

No , print -1

Linear Search : Code

```
1   #include<stdio.h>
2
3   int linearSearch(int array[],int n,int target){
4       int found = -1 ;
5       for(int i=0 ; i<n ; i++) {
6           if(array[i]==target){
7               found = i;
8               break;
9           }
10      }
11      return found ;
12  }
13  int main() {
14      int array[] = {3,9,6,1,2};
15      printf("%d",linearSearch(array,5,1));
16      return 0 ;
17  }
```



Binary Search

Algorithm

- If an array is **sorted**
- Check whether the middle element is equal to target.

  If so, it's done.
- Otherwise, determine which half of array the target should be in (if target < mid then it's on 1st half , else it's in the 2nd half)
- Repeat …

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 20

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

#	low	high	mid

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 20

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

#	low	high	mid
1	0	12	6

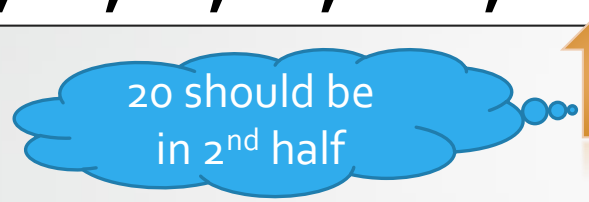
Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 20

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }



20, 21, 25, 34, 39, 41

#	low	high	mid
1	0	12	6

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 20

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

20 should be in 2nd half

20, 21, 25, 34, 39, 41

#	low	high	mid
1	0	12	6
2	7	12	9

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 20

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

20 should be in 2nd half

20, 21, 25, 34, 39, 41

20 should be in 1st half

20, 21

#	low	high	mid
1	0	12	6
2	7	12	9

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 20

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

20 should be in 2nd half

20, 21, 25, 34, 39, 41

20 should be in 1st half

20, 21

#	low	high	mid
1	0	12	6
2	7	12	9
3	7	8	7

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 20

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

20 should be in 2nd half

20, 21, 25, 34, 39, 41

20 should be in 1st half

20, 21

20 found

return 7

#	low	high	mid
1	0	12	6
2	7	12	9
3	7	8	7

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 6

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

#	low	high	mid

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 6

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

#	low	high	mid
1	0	12	6

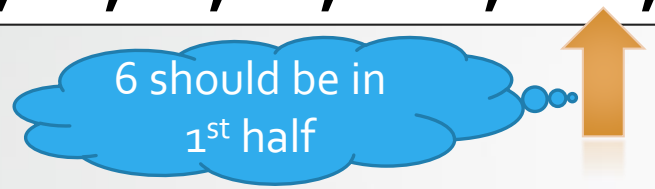
Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 6

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }



1, 3, 4, 7, 9, 12

#	low	high	mid
1	0	12	6

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 6

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

6 should be in 1st half

1, 3, 4, 7, 9, 12

#	low	high	mid
1	0	12	6
2	0	5	2

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 6

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

6 should be in 1st half

1, 3, 4, 7, 9, 12

6 should be in 2nd half

7, 9, 12

#	low	high	mid
1	0	12	6
2	0	5	2

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 6

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

6 should be in 1st half

1, 3, 4, 7, 9, 12

6 should be in 2nd half

7, 9, 12

#	low	high	mid
1	0	12	6
2	0	5	2
3	3	5	4

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 6

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

6 should be in 1st half

1, 3, 4, 7, 9, 12

6 should be in 2nd half

7, 9, 12

6 should be in 1st half

7

#	low	high	mid
1	0	12	6
2	0	5	2
3	3	5	4

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 6

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

6 should be in 1st half

1, 3, 4, 7, 9, 12

6 should be in 2nd half

7, 9, 12

6 should be in 1st half

7

#	low	high	mid
1	0	12	6
2	0	5	2
3	3	5	4
4	3	3	3

Binary Search : example

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 6

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	4	7	9	12	17	20	21	25	34	39	41

x[] = {1, 3, 4, 7, 9, 12, 17, 20, 21, 25, 34, 39, 41 }

6 should be in 1st half

1, 3, 4, 7, 9, 12

6 should be in 2nd half

7, 9, 12

6 should be in 1st half

7

6 is not found ☹

return -1

#	low	high	mid
1	0	12	6
2	0	5	2
3	3	5	4
4	3	3	3

Binary Search : Code

```
1   #include<stdio.h>
2
3   int binarySearch(int array[] , int target , int low , int high) {
4       int found =-1 , mid ;
5
6       while( low<=high && found == -1) {
7           mid = (low + high) / 2 ;
8           if (target == array[mid]) {
9               found = mid ;
10          }
11          else if (target < array[mid]) {
12              high = mid - 1 ;
13          }
14          else if (target > array[mid]) {
15              low = mid + 1 ;
16          }
17      }
18
19      return found ;
20  }
21
22  int main() {
23      int array[] = {1,2,3,4,5,6,7,8,9,10};
24      printf("%d",binarySearch(array,5,0,9));
25      return 0 ;
26  }
```

If array is not empty … And still didn't found the target

Target should be in 1st half

Target should be in 2nd half

Binary Search : Code

Recursive Solution

```
1   #include<stdio.h>
2
3   int binarySearch(int array[],int target,int low , int high) {
4       if( low<=high) {
5           int mid = (high + low) / 2;
6           if (target == array[mid]){
7               return mid;
8           }
9           else if (target < array[mid]) {
10              return binarySearch(array, target , low, mid-1);
11          }
12          else {
13              return binarySearch(array, target , mid+1, high);
14          }
15      }
16      else {
17          return -1;
18      }
19  }
20
21  int main() {
22      int array[] = {1,2,3,4,5,6,7,8,9,10};
23      printf("%d",binarySearch(array 5,0,9 ));
24      return 0 ;
25  }
```

If array is not empty

Target should be in 1st half

Target should be in 2nd half

Sheet4-Arrays : Q5.3

5.3 Trace the following set of numbers for the values 12, 34 using binary search:

2 5 7 8 10 12 15 20 21 27 33 40 45

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 12

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

x[]={2, 5, 7, 8, 10, 12, 15, 20, 21, 27, 33, 40, 45}

#	low	high	mid

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 12

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

#	low	high	mid
1	0	12	6

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 12

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

x[]={2, 5, 7, 8, 10, 12, 15, 20, 21, 27, 33, 40, 45}

12 should be in 1st half

2, 5, 7, 8, 10, 12

#	low	high	mid
1	0	12	6

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 12

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

x[]={2, 5, 7, 8, 10, 12, 15, 20, 21, 27, 33, 40, 45}

12 should be in 1st half

2, 5, 7, 8, 10, 12

#	low	high	mid
1	0	12	6
2	0	5	2

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 12

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

12 should be in 1st half

2, 5, 7, 8, 10, 12

12 should be in 2nd half

8, 10, 12

#	low	high	mid
1	0	12	6
2	0	5	2

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 12

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

12 should be in 1st half

2, 5, 7, 8, 10, 12

12 should be in 2nd half

8, 10, 12

#	low	high	mid
1	0	12	6
2	0	5	2
3	3	5	4

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 12

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

12 should be in 1st half

2, 5, 7, 8, 10, 12

12 should be in 2nd half

8, 10, 12

12 should be in 2nd half

12

#	low	high	mid
1	0	12	6
2	0	5	2
3	3	5	4

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 12

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

12 should be in 1st half

2, 5, 7, 8, 10, 12

12 should be in 2nd half

8, 10, 12

12 should be in 2nd half

12

#	low	high	mid
1	0	12	6
2	0	5	2
3	3	5	4
4	5	5	5

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 12

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

x[]={2, 5, 7, 8, 10, 12, 15, 20, 21, 27, 33, 40, 45}

12 should be in 1st half

2, 5, 7, 8, 10, 12

12 should be in 2nd half

8, 10, 12

12 should be in 2nd half

12

12 is found

return 5

#	low	high	mid
1	0	12	6
2	0	5	2
3	3	5	4
4	5	5	5

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 34

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

x[]={2, 5, 7, 8, 10, 12, 15, 20, 21, 27, 33, 40, 45}

#	low	high	mid

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 34

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

#	low	high	mid
1	0	12	6

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 34

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

x[]={2, 5, 7, 8, 10, 12, 15, 20, 21, 27, 33, 40, 45}

34 should be in 2nd half

20, 21, 27, 33, 40, 45

#	low	high	mid
1	0	12	6

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 34

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

34 should be in 2nd half

20, 21, 27, 33, 40, 45

#	low	high	mid
1	0	12	6
2	7	12	9

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 34

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

x[]={2, 5, 7, 8, 10, 12, 15, 20, 21, 27, 33, 40, 45}

34 should be in 2nd half

20, 21, 27, 33, 40, 45

34 should be in 2nd half

33, 40, 45

#	low	high	mid
1	0	12	6
2	7	12	9

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 34

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

34 should be in 2nd half

20, 21, 27, 33, 40, 45

34 should be in 2nd half

33, 40, 45

#	low	high	mid
1	0	12	6
2	7	12	9
3	10	12	11

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 34

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

34 should be in 2nd half

20, 21, 27, 33, 40, 45

34 should be in 2nd half

33, 40, 45

34 should be in 1st half

33

#	low	high	mid
1	0	12	6
2	7	12	9
3	10	12	11

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 34

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

x[]={2, 5, 7, 8, 10, 12, 15, 20, 21, 27, 33, 40, 45}

34 should be in 2nd half

20, 21, 27, 33, 40, 45

34 should be in 2nd half

33, 40, 45

34 should be in 1st half

33

#	low	high	mid
1	0	12	6
2	7	12	9
3	10	12	11
4	10	10	10

Sheet4-Arrays : Q5.3

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

Target = 34

0	1	2	3	4	5	6	7	8	9	10	11	12
2	5	7	8	10	12	15	20	21	27	33	40	45

x[]={2, 5, 7, 8, 10, 12, 15, 20, 21, 27, 33, 40, 45}

34 should be in 2nd half

20, 21, 27, 33, 40, 45

34 should be in 2nd half

33, 40, 45

34 should be in 1st half

33

34 is not found ☹

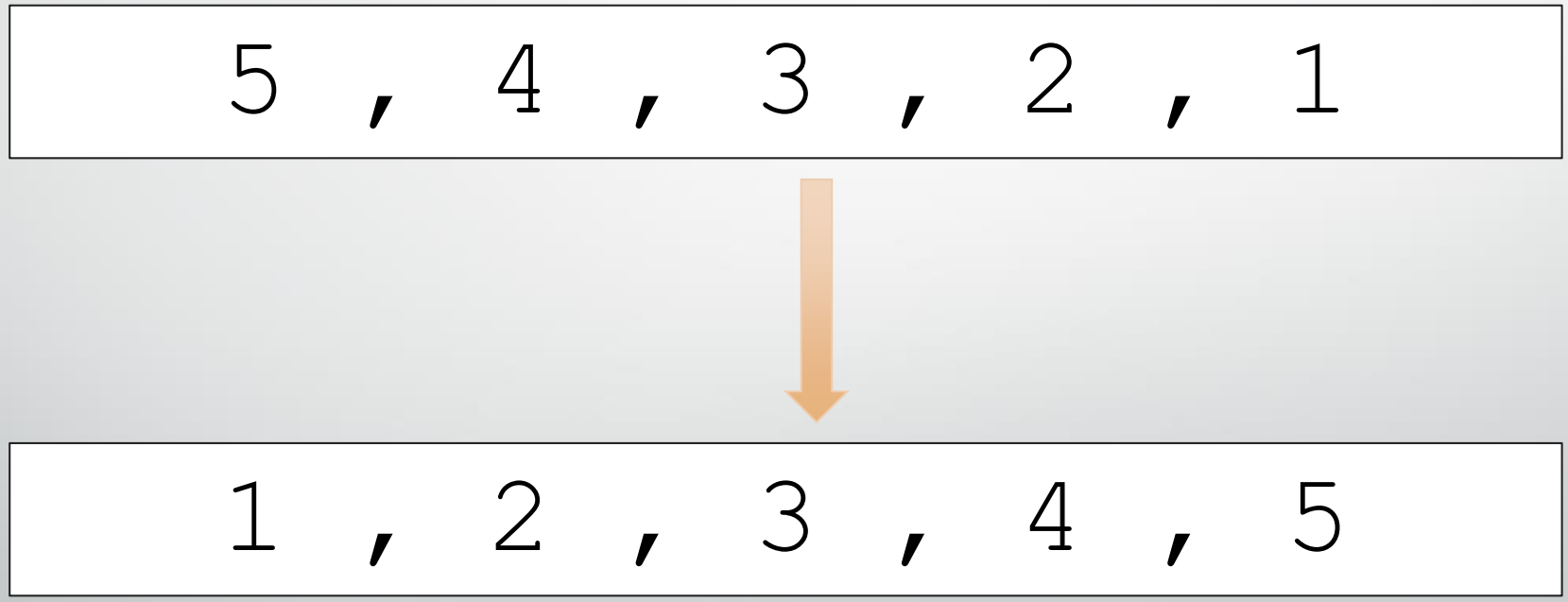
return -1

#	low	high	mid
1	0	12	6
2	7	12	9
3	10	12	11
4	10	10	10

PART2

Sorting

Sorting takes an unordered collection and makes it an ordered one.



Sorting Algorithms

Sorting Algorithms examples :

- **Bubble sort** ⬅
- **Selection Sort** ⬅
- Insertion Sort
- Merge Sort
- And more ….

Bubble Sort : example 1

Pass 1

5 , 4 , 3 , 2 , 1
4 , **5 , 3** , 2 , 1
4 , 3 , **5 , 2** , 1
4 , 3 , 2 , **5 , 1**
4 , 3 , 2 , 1 , 5

Pass 2

4 , 3 , 2 , 1 , 5
3 , **4 , 2** , 1 , 5
3 , 2 , **4 , 1** , 5
3 , 2 , 1 , 4 , 5

Pass 3

3 , 2 , 1 , 4 , 5
2 , **3 , 1** , 4 , 5
2 , 1 , 3 , 4 , 5

Pass 4

2 , 1 , 3 , 4 , 5
1 , 2 , 3 , 4 , 5

Number of Comparisons =

4+3+2+1 = 10

Bubble Sort Algorithm

Algorithm

```
For i=1 … n
  For j=0 … n-i
    if(array[j] > array[j+1])
      swap them
```

Bubble Sort : Code

```
1   #include<stdio.h>
2
3   void swap(int &x , int &y) {
4       int temp = x ;
5       x = y ;
6       y = temp ;
7   }
8
9   void bubbleSort(int array[],int n){
10      for(int i=1 ; i<n ; i++) {
11          for(int j=0 ; j<n-i ; j++) {
12              if(array[j]>array[j+1]) {
13                  swap(array[j],array[j+1]);
14              }
15          }
16      }
17  }
18
19  int main() {
20      int array[] = {5,4,3,2,1};
21      bubbleSort(array,5);
22      for(int i =0 ; i<5 ; i++){
23          printf("%d ",array[i]);
24      }
25      return 0 ;
26  }
```

Bubble Sort : example 2

Pass 1

1 , 2 , 3 , 9 , 4
1 , **2 , 3** , 9 , 4
1 , 2 , **3 , 9** , 4
1 , 2 , 3 , **9 , 4**
1 , 2 , 3 , 4 , 9

Pass 2

1 , 2 , 3 , 4 , 9
1 , **2 , 3** , 4 , 9
1 , 2 , **3 , 4** , 9
1 , 2 , 3 , 4 , 9

Pass 3

1 , 2 , 3 , 4 , 9
1 , **2 , 3** , 4 , 9
1 , 2 , 3 , 4 , 9

Pass 4

1 , 2 , 3 , 4 , 9
1 , 2 , 3 , 4 , 9

Number of Comparisons =
4+3+2+1 = 10

How can we optimize it ?

Bubble Sort Algorithm Optimized

```
For i=1 … n
  swapped = 0;
    For j=0 … n-i
     if(array[j] > array[j+1])
       swap them;
       swapped = 1;
  if(swapped == 0)
    break;
```

Algorithm Improved

Bubble Sort : example 2 optimized

Pass 1

1	**2**	3	9	4
1	**2**	**3**	9	4
1	2	**3**	**9**	4
1	2	3	**9**	**4**
1	2	3	4	9

Pass 2

1	**2**	3	4	9
1	**2**	**3**	4	9
1	2	**3**	**4**	9
1	2	3	4	9

Number of Comparisons =

4+3 = 7

Bubble Sort : Code Optimized

```
1   #include<stdio.h>
2   void swap(int &x , int &y) {
3       int temp = x ;
4       x = y ;
5       y = temp ;
6   }
7
8   void bubbleSort(int array[],int n){
9       for(int i=1 ; i<n ; i++) {
10          int swapped = 0;
11          for(int j=0 ; j<n-i ; j++) {
12              if(array[j]>array[j+1]) {
13                  swap(array[j],array[j+1]);
14                  swapped = 1 ;
15              }
16          }
17          if(swapped == 0) { break; }
18      }
19  }
20
21  int main() {
22      int array[] = {5,4,3,2,1};
23      bubbleSort(array,5);
24      for(int i =0 ; i<5 ; i++){
25          printf("%d ",array[i]);
26      } return 0 ;
27  }
```

Selection Sort

Algorithm

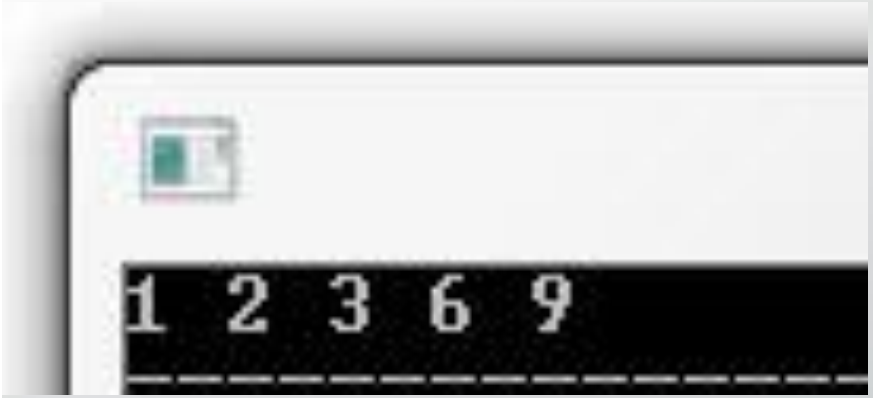
- ✓ Find the smallest value in the list
- ✓ Switch it with the value in the first position
- ✓ Find the next smallest value in the list
- ✓ Switch it with the value in the second position
- ✓ Repeat until all values are in their proper places

Selection Sort : example

3 , 9 , 6 , **1** , 2
1 , **9** , 6 , 3 , **2**
1 , 2 , **6** , **3** , 9
1 , 2 , 3 , **6** , 9
1 , 2 , 3 , 6 , 9

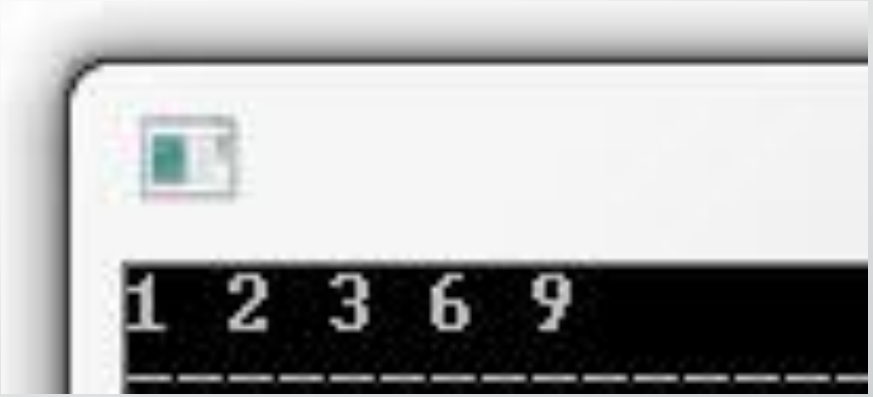
Selection Sort : Code

```
1   #include<stdio.h>
2   void swap(int &x , int &y) {
3       int temp = x ;
4       x = y ;
5       y = temp ;
6   }
7
8   void selectionSort(int array[],int n){
9       for(int i=0 ; i<n ; i++) {
10          int minIndex = i ;
11          for(int j=i+1 ; j<n ; j++) {
12              if(array[j]<array[minIndex]) {
13                  minIndex = j ;
14              }
15          }
16          swap(array[i], array[minIndex]);
17      }
18  }
19
20  int main() {
21      int array[] = {3,9,6,1,2};
22      selectionSort(array,5);
23      for(int i =0 ; i<5 ; i++){
24          printf("%d ",array[i]);
25      } return 0 ;
26  }
```



Selection Sort : Code Optimized

```
1   #include<stdio.h>
2   void swap(int &x , int &y) {
3       int temp = x ;
4       x = y ;
5       y = temp ;
6   }
7   void selectionSort(int array[],int n){
8       for(int i=0 ; i<n ; i++) {
9           int minIndex = i ;
10          for(int j=i+1 ; j<n ; j++) {
11              if(array[j]<array[minIndex]) {
12                  minIndex = j ;
13              }
14          }
15          if(minIndex != i){
16          swap(array[i], array[minIndex]);
17          }
18      }
19  }
20  int main() {
21      int array[] = {3,9,6,1,2};
22      selectionSort(array,5);
23      for(int i=0 ; i<5 ; i++){
24          printf("%d ",array[i]);
25      }
26      return 0 ;
27  }
```



Selection Sort : example optimized

3 , 9 , 6 , **1** , 2
1 , **9** , 6 , 3 , **2**
1 , 2 , **6** , **3** , 9
1 , 2 , 3 , 6 , 9

END of Section

Thank You ☺

Eng. Mahmoud Osama Radwan
mhmoudko@msn.com